보안, 인프라, AI 보안 엔지니어

기민수

미디어솔루션에서 1년 11개월 동안 병역특례 산업기능요원으로 근무하며 대형병원, 대형 컨벤션센터, 5성급 호텔 등 4개 기업 고객의 키오스크 CMS와 운영 서버를 개발하고 운영했습니다. 고객 시스템에서 일어난 침해사고 두 건을 현장에서 대응했고, 웹 애플리케이션을 AWS에 다시 구축하며 외부에서 접근할 수 있는 경로를 줄였습니다. 지금은 개인 프로젝트로 AI 관련 보안 도구를 만들고 있습니다. LLM과 MCP 통신에서 개인정보와 기밀이 새어 나가지 않게 막는 프록시, 정적분석과 LLM 검증을 묶은 취약점 스캐너가 그 예입니다.

사례 원장	7건
01	**고객사 웹 서버 침해사고 두 건 대응과 AWS 재구축** + 서버: AWS EC2와 RDS로 재구축
02	**대형병원 키오스크 CMS의 권한 결함 수정과 보안점검 대응** + 권한 부여 순서: 요청, 세션 검증, 권한 부여
03	**ETK-Scanner** + 위조 토큰 요청: HTTP 401로 차단
04	**Mcp-Guard** + 민감 우회 사례 탐지: 60건 중 56건
05	**EvidScope** + 지속 구간 접수: 600건 모두
06	**서버 7대 통합과 여러 겹의 백업 설계** + 복구 수단: 변경 제한 백업, AWS 서버 이미지, MSSQL 복제
07	**5성급 호텔 서버의 디스크 고갈 장애와 재발 방지** + 디스크 정리: cron으로 주기 실행

경력

미디어솔루션연구개발팀 사원

2023.04–2025.03

미디어솔루션은 직원 50명 이하의 소기업입니다. 병역특례 산업기능요원으로 1년 11개월 근무하며 대형병원, 대형 컨벤션센터, 5성급 호텔 등 4개 기업 고객의 키오스크 CMS와 운영 인프라를 맡았습니다.

CASE-01 • 보안 사례 • 2023.04–2025.03

고객사 웹 서버 침해사고 두 건 대응과 AWS 재구축

KISA 침해 통보와 서비스 중단 신고로 시작된 두 사고에 현장에서 대응하고, 무결성을 확인한 데이터만 AWS로 옮겨 서버를 다시 세웠습니다.

역할

현장 대응, 로그 분석, 복구와 AWS 재구축, 고객 보고

직무

보안, 인프라

대응 흐름

01 **통보와 신고**
1차는 KISA의 침해 통보, 2차는 서비스가 멈췄다는 고객 신고로 시작했습니다

02 **접근 되찾기**
시스템 접근과 관리자 계정을 되찾아 로그와 프로세스를 볼 수 있게 했습니다

03 **원인 추적**
IIS 로그를 따라가 응용프로그램 풀 안의 악성 실행을 찾았습니다

04 **데이터 분리**
MSSQL 데이터를 격리된 PC로 옮겨 무결성을 확인했습니다

05 **다시 세우기**
AWS EC2와 RDS에 새로 세우고 운영 IP만 열었습니다

해결한 문제

문제	쓴 것	효과
백업 복원이 중간에 멈춤	멈춘 지점을 따라가 업로드 파일을 점검	**검증 없이 저장된 과대 크기 악성 이미지를 분리하고 복원을 마침**
보안 비용을 걱정하는 고객사	비용과 걸리는 시간을 그래프와 도형으로 정리	**클라우드 이관과 주기적인 서버 관리에 합의**
VPN을 쓸 수 없는 구간	서비스 포트와 출발지 IP를 한정	**외부에 열린 범위를 줄임**

전과 후

서버	− ~~고객사 Windows 서버와 오래된 ASP.NET~~
	+ **AWS EC2와 RDS로 재구축**
관리 접근	+ **운영 IP 허용목록만 열고 RDP 차단**
업로드 파일	− ~~검증 없이 저장~~
	+ **파일과 입력 무결성 검증 뒤 저장**
복구	+ **사고 인지부터 복구까지 4일**

같은 원인의 재발	+ 재직 중 확인되지 않음

배경

상황

미디어솔루션은 병원, 컨벤션센터, 호텔 같은 기업 고객의 키오스크 CMS와 운영 서버를 맡는 소기업이었습니다. 한 고객사의 웹 서버는 오래된 ASP.NET 위에서 돌고 있었고, 이 서버에서 침해사고가 두 번 일어났습니다. 두 사고 모두 제가 현장에 나가 대응했습니다.

어떻게 알았나

1차 사고는 KISA가 고객사 서버의 침해를 통보해 알게 됐습니다.

2차 사고는 서비스가 멈췄다는 고객 신고로 시작됐습니다. 서버의 CPU와 메모리가 비정상적으로 점유돼 있었고 공격자가 설치한 가상화 환경이 있어, 서버 자원을 몰래 쓰는 크립토마이닝을 의심했습니다.

무엇부터, 왜

무엇이 남아 있는지 보려면 시스템 접근과 관리자 계정부터 되찾아야 했습니다. 이 단계 없이는 로그도 프로세스도 확인할 수 없었기 때문입니다.

고객에게 조치가 필요한 이유를 설명할 때는 기존 .NET Framework 버전과 업데이트 버전의 공개 CVE 수, 사고 빈도를 근거로 삼았습니다. 이 자료로 AI를 활용해 위험 감소 추정치를 만들었고 출처를 함께 남겼습니다.

내가 한 것

- 현장에서 시스템 접근과 관리자 계정을 되찾고, IIS 로그를 따라가 응용프로그램 풀 안의 악성 실행을 루트킷으로 판단했습니다.
- 구형 미들웨어와 ASP.NET 관련 취약점을 점검하고 조치했습니다.
- MSSQL 데이터는 내부망의 격리된 PC로 옮겨 무결성을 확인한 뒤 분리했습니다.
- 2차 사고 복구 중 MSSQL 백업 복원이 멈춘 원인을 찾아, 업로드 검증 없이 저장된 과대 크기의 악성 이미지 파일을 분리하고 제거했습니다.
- 웹 애플리케이션을 AWS EC2에 새로 세우고 DB를 RDS로 옮겼습니다. 보안그룹은 운영 IP 허용목록만 열고 RDP 접근을 막았습니다.

쓴 기술

IIS 로그	응용프로그램 풀 안의 악성 실행 추적
MSSQL	격리된 PC에서 데이터 무결성 확인과 복원
AWS EC2, RDS	오래된 고객사 서버를 대신할 재구축 환경
보안그룹	운영 IP 허용목록과 RDP 차단

함께 고친 과정

기술 담당자에게는 취약한 구성과 코드 문제를, 경영진에게는 서비스 중단과 복구 비용 관점의 권고를 따로 설명했습니다.

파트너사 담당자는 보안 조치에 드는 비용과 재발을 걱정했고, 서비스 복구 말고는 관심이 크지 않았습니다. 그래서 비용과 걸리는 시간을 그래프와 도형으로 보여 줬습니다. 설명을 들은 고객은 클라우드 이관과 주기적인 서버 관리에 합의했습니다.

확인과 재발 방지

파일과 입력 무결성 검증을 강화한 서버를 다시 배포했습니다. VPN을 쓸 수 없는 구간은 서비스 포트와 출발지 IP를 한정했습니다. AWS로 옮긴 뒤 제가 재직하는 동안 같은 원인의 공격은 다시 확인되지 않았습니다.

CASE-02 • 보안 사례 • 2023.04–2025.03

대형병원 키오스크 CMS의 권한 결함 수정과 보안점검 대응

대형병원 키오스크를 원격으로 관리하는 CMS를 혼자 만들어 운영했고, 세션 검증보다 권한 부여가 먼저 일어나던 결함을 고쳐 병원 보안점검을 통과했습니다.

역할

CMS 단독 설계, 개발, 운영과 보안점검 대응

직무

보안, 개발

대응 흐름

01 **요청**
관리자나 운영자가 CMS에 로그인해 요청을 보냅니다

02 **세션 검증**
Redis에서 세션을 먼저 확인합니다

03 **권한 부여**
검증이 끝난 세션에만 역할 권한을 붙입니다

04 **원격 제어**
WebSocket으로 키오스크 화면을 보고 명령을 보냅니다

05 **배포**
예약과 그룹 단위로 배포하고 문제가 생기면 롤백합니다

해결한 문제

문제	쓴 것	효과
세션 검증 전에 임시 권한이 붙는 구간	권한 부여를 세션 검증 뒤로 옮김	**검증되지 않은 세션이 권한을 쥐는 구간을 없앰**
병원의 계정 정책 요구	정기 변경, 실패 횟수 제한, 재사용 금지, 계정 잠금	**납품한 CMS 전체가 병원 보안점검 통과**
장애마다 현장으로 가야 하는 운영	WebSocket 화면 미러링과 원격 명령	**원격으로 풀 수 있는 장애는 CMS에서 처리**
외부 인증기관 접속이 막힌 망분리 환경	NICE CPClient, Electron 앱, IIS와 .NET 로컬 인증 서버를 콜백으로 연결	**망분리 안에서 본인인증 흐름 배포**

전과 후

권한 부여 순서	− ~~요청, 임시 권한 부여, 세션 검증~~
	+ 요청, 세션 검증, 권한 부여
계정 정책	+ 정기 변경, 실패 횟수 제한, 재사용 금지, 계정 잠금

키오스크 장애 처리	− 담당자가 현장으로 이동
	+ 원격으로 풀 수 있는 장애는 CMS로 처리

배경

상황

고객은 4만 평 규모의 대형병원이었습니다. 병원 내부망의 키오스크 약 200대는 장애가 날 때마다 담당자가 현장으로 가야 했습니다. 이 부담을 줄이려고 원격 관리 CMS를 혼자 만들어 운영했습니다. CMS에는 화면 미러링과 원격 명령, 예약 배포를 넣었습니다.

납품하는 CMS는 병원 보안점검을 통과해야 했고, 권한 분리와 계정 정책을 병원 기준에 맞춰야 했습니다.

어떻게 알았나

병원 보안 요구에 맞춰 인증과 권한 구조를 고치던 중에, Redis에서 세션 검증이 끝나기 전에 임시 권한이 먼저 붙는 구간을 찾았습니다. 요청 타이밍에 따라 검증되지 않은 세션이 권한을 쥘 수 있는 구조였습니다.

내가 한 것

- 권한 부여를 세션 검증 뒤로 옮겨 처리 순서를 바로잡았습니다.
- 서버 개인키로 서명하는 JWT 인증과 bcrypt 비밀번호 해시, 관리자와 운영자 역할을 나눈 RBAC를 구현했습니다.
- 90일마다 비밀번호를 바꾸게 하고, 로그인 실패 횟수 제한과 재사용 금지, 계정 잠금 정책을 적용했습니다.
- ORM으로 쿼리를 만들어 SQL을 직접 조립하는 곳을 줄였습니다.
- 연동 문제는 브라우저 개발자 도구의 Network 탭에서 엔드포인트와 요청 내용, Authorization 헤더의 JWT를 확인하며 분석했습니다.

갖춘 것

화면 미러링

키오스크 화면을 CMS에서 그대로 봅니다

원격 명령

재시작 같은 명령을 현장에 가지 않고 보냅니다

예약 배포와 롤백

그룹 단위로 예약 배포하고 중단점으로 되돌립니다

역할 권한

관리자와 운영자의 권한을 나눕니다

쓴 기술

TypeScript, Angular, Express	CMS 화면과 API
MariaDB, Redis	데이터 저장과 세션 확인
WebSocket	화면 미러링과 원격 명령
JWT, bcrypt, RBAC	인증, 비밀번호 해시, 역할 권한 분리

확인과 재발 방지

납품한 CMS 전체가 병원 보안점검을 통과했습니다.

CASE-03 • 도구 사례 • 2026.07

ETK-Scanner, 비용 상한이 있는 LLM 소스코드 취약점 진단

정적분석으로 의심 구간을 먼저 좁히고 LLM 검증은 필요한 곳에만 쓰는 취약점 스캐너를 만들어, 실습 백엔드와 오픈소스의 권한 문제를 찾았습니다.

역할	직무	저장소
파이프라인 설계, 구현, 검증	보안, AI 보안	github.com/ki0915/etk-scanner

처리 흐름

01 후보 좁히기
정적분석으로 보안 판단 함수와 위험 지점을 후보로 뽑습니다. LLM 비용이 들지 않습니다

02 저비용 분류
저비용 모델이 후보마다 한 번 분류합니다

03 고비용 검증
남은 소수 후보만 고비용 모델이 문맥을 여러 번 확인합니다

04 비용 회계
호출마다 비용을 기록하고 예산에 닿으면 멈춥니다

05 보고
SARIF로 GitHub 보안 탭과 CI job 결과에 연결합니다

해결한 문제

문제	쓴 것	효과
큰 저장소에서 검사 조각이 불어나 비용이 커짐	테스트와 문서 폴더를 검사 대상에서 제외	**검사 조각 수를 줄임**
저비용 분류가 음성이라 하면 상위 검증이 막힘	재현율 우선 분류, 응답을 못 읽으면 후보로 유지	**놓치던 사례가 다시 검증 단계로 넘어감**
재현된 결함과 보고할 만한 결함이 섞임	악용 가능성 확인 단계와 보안 영향 필터	**기능 버그를 자동으로 걸러 냄**
이미 알려진 취약점을 제보 직전까지 감	OSV와 GHSA 중복 확인 단계 추가	**중복 제보를 막음**

적용 전과 후

위조 토큰 요청	− ~~HTTP 200, 보호 데이터 노출~~ + HTTP 401로 차단
SecurityEval 취약 코드	+ 121개 중 111개 탐지

배경

풀려던 문제

위험 함수 호출만 찾는 규칙 기반 도구는 권한 우회나 검증 우회처럼 위험 함수 없이 생기는 결함을 잡지 못합니다. 모든 함수를 LLM에 보내면 비용 때문에 CI에 붙이기 어렵습니다. 비용 대비 효율이 좋은 취약점 진단 도구를 만들고자 시작했습니다.

고른 설계와 이유

비용이 적은 단계부터 거르도록 정적분석, 저비용 모델 분류, 고비용 모델 검증 순서로 나눴습니다.

후보에는 eval 같은 위험 함수와 함께 validate나 auth처럼 보안 판단을 하는 함수를 넣었습니다. 권한 우회나 검증 우회는 위험 함수 호출 없이도 생기기 때문입니다.

직접 설계하고 만든 것

- Python 코드는 AST로, JavaScript와 TypeScript는 정규식으로 보안 판단 함수를 후보로 뽑습니다. 이 단계는 LLM 비용이 들지 않습니다.
- 저비용 모델이 후보마다 한 번 분류하고, 남은 소수만 고비용 모델이 여러 차례 문맥을 확인하며 검증합니다.
- 모든 LLM 호출의 비용을 기록하고, 정한 예산에 닿으면 분석을 멈춥니다.
- 결과를 SARIF로 내보내 GitHub 보안 탭에 올리고, GitHub Actions에서 PR과 push마다 정적 모드를 돌려 검사가 실패하면 job을 실패로 처리합니다.

기능

보안 판단 함수 탐지

Python 코드는 AST로, JavaScript와 TypeScript 코드는 정규식으로 읽어 보안 판단이 들어간 함수를 후보로 뽑습니다.

공개 SecurityEval 데이터의 취약 코드 121개에 스캐너를 실행해 111개를 탐지했습니다.

단계별 LLM 검증

저비용 모델이 후보마다 한 번 질의해 취약점 가설과 신뢰도를 매기고, 남은 소수 후보만 고비용 모델이 여러 차례 질의하며 검증합니다.

비용 기록과 예산 상한

모든 LLM 호출의 토큰과 비용을 누적해 기록하고, 정한 예산에 닿으면 분석을 멈춥니다.

SARIF 출력

탐지 결과의 CWE, 심각도, 파일과 행 위치를 SARIF 2.1.0으로 변환해 GitHub 보안 탭에 올릴 수 있게 합니다.

CI 연동

GitHub Actions에서 PR과 push에는 기본 정적 모드를 돌리고 LLM 모드는 수동으로 실행하며, 검사가 실패하면 마지막 job을 실패로 처리합니다.

인가 로직 분석

소스 코드에서 권한을 확인하는 처리를 따라가며 문제가 생기는 원인을 좁혔습니다.

재현 자료와 원인 분석 전달

문제를 재현하는 자료와 원인 분석을 정리해 유지관리자에게 보냈습니다.

유지관리자가 영향을 확인하고 수정할 예정이라고 회신했습니다.

시연

terminal

```
$ python -m unittest discover -s tests
.........................................

----------------------------------------------------------------------

Ran 45 tests in 2.152s



OK
```

python -m unittest discover -s tests 실행 결과

terminal

```
$ python -m unittest discover -s tests
.........................................

----------------------------------------------------------------------

Ran 45 tests in 2.152s



OK
```

python -m unittest discover -s tests 실행 결과

기술

Python ast	보안 판단 함수 후보 추출
Claude Haiku, Sonnet	단계별 분류와 검증

SARIF 2.1.0, GitHub Actions	보안 탭 업로드와 CI 실패 처리
OSV.dev API	이미 알려진 취약점인지 확인

함께 쓰고 고친 과정

이 스캐너를 활용해 한 오픈소스 프로젝트의 인가 처리 문제를 소스 수준에서 분석했습니다. 재현 자료와 원인 분석을 유지관리자에게 보냈고, 영향 확인과 수정 예정 회신을 받았습니다. 공개 전이라 프로젝트 이름과 세부 내용은 적지 않습니다.

검증 방법

직접 만든 TypeScript 실습 백엔드를 검사하자 소스에 박힌 JWT 서명 키가 잡혔습니다. 이 키를 알면 누구나 토큰을 위조할 수 있습니다. 키를 환경변수로 옮기기 전과 후를 실제 HTTP 서버로 비교했습니다.

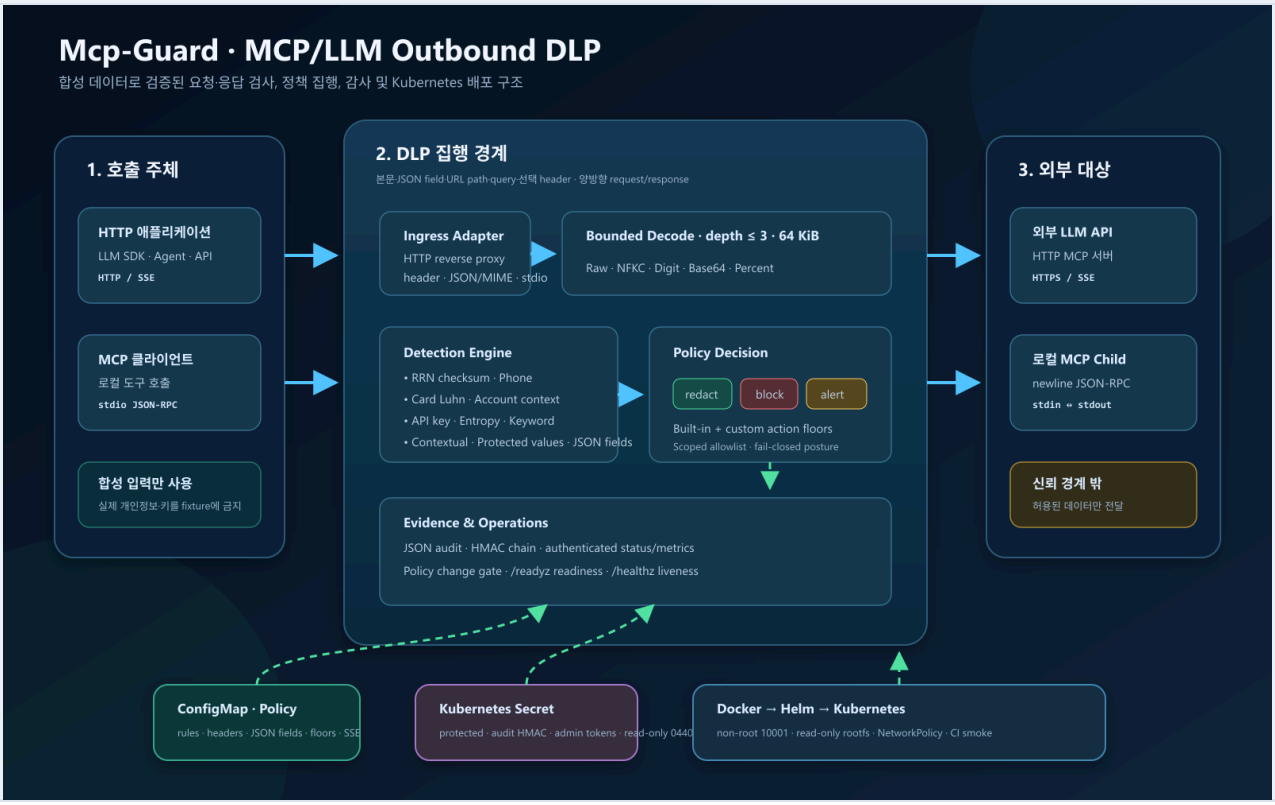
공개 SecurityEval 데이터의 취약 코드에도 스캐너를 돌려 몇 개를 찾는지 확인했습니다.

CASE-04 • 도구 사례 • 2026.07

Mcp-Guard, LLM과 MCP 통신의 개인정보 유출 방지 프록시

LLM과 MCP 도구로 나가는 요청에서 주민등록번호와 API 키 같은 민감 정보를 찾아 가리거나 막는 프록시를 만들었습니다.

역할	직무	저장소
정책과 탐지 설계, 구현, 검증	보안, AI 보안	github.com/ki0915/Mcp-Guard



Mcp-Guard 아키텍처

처리 흐름

01 받기
HTTP 리버스 프록시와 stdio 어댑터가 요청과 응답을 받습니다

02 풀어 보기
NFKC 정규화, 숫자 사이 공백과 보이지 않는 문자 제거, 크기를 제한한 Base64 디코딩

03 찾기
주민등록번호, 카드, 계좌, 시크릿, 조직 기밀 탐지기가 위치를 찾습니다

04 판정
정책에 따라 가리기, 막기, 경보 가운데 하나를 고르고, 판단이 어려우면 막습니다

05 기록
원래 값을 뺀 감사 로그와 Prometheus 지표를 남깁니다

해결한 문제

문제	→ 쓴 것	→ 효과
HTTP만 보는 프록시는 stdio MCP 서버에 닿지 못함	줄 단위 JSON-RPC를 검사하는 stdio 어댑터	**두 통신 경로에 같은 정책을 적용**

문제	쓴 것	효과
띄어 쓴 번호, 전각 숫자, Base64로 감싼 값이 통과	원문 위치를 유지하는 정규화 뒤 검사	**합성 우회 사례 탐지가 크게 늘어남**
실제 기밀값이 설정 이력에 남을 위험	기밀값은 Kubernetes Secret에만 두고 해시로 대조	**ConfigMap과 Helm 이력에 원래 값이 남지 않음**
원문을 그대로 보내는 경보 설정으로 정책이 약해질 위험	주민등록번호, 카드, 시크릿은 경보로 낮출 수 없게 함	**고위험 값은 항상 가리거나 막음**

적용 전과 후

민감 우회 사례 탐지	− ~~60건 중 10건~~ + **60건 중 56건**
정상 사례 오탐	= **30건 중 0건** 그대로 유지
HTTP 프록시 추가 지연	+ **평균 3.10ms**

배경

풀려던 문제

생성형 AI를 업무에 쓰면 사람이 붙여 넣은 문서나 로그에 개인정보와 시크릿이 섞이기 쉽습니다. MCP 서버는 표준 입출력으로 JSON-RPC 메시지를 주고받는 경우가 많습니다. 그래서 네트워크 경계에 둔 HTTP 프록시만으로는 내용을 볼 수 없습니다.

문제를 확인한 방법

처음 버전은 원문 그대로만 검사했고, 띄어 쓴 주민등록번호나 전각 숫자, Base64로 감싼 값이 그대로 통과했습니다. 우회 기법을 넣은 합성 데이터로 그 차이를 쟀습니다.

고른 설계와 이유

HTTP 프록시와 stdio 어댑터에 같은 정책을 적용해 두 경로의 판단이 같게 했습니다. 검사는 원문 위치를 유지한 파생 문자열로 하고, 마스킹은 원래 입력의 해당 범위에 적용했습니다. 판단이 어려우면 막고, 감사 로그에는 원래 값을 남기지 않게 했습니다.

직접 설계하고 만든 것

- HTTP 리버스 프록시는 요청과 응답 본문, URL, SSE 이벤트를 검사하고, stdio 어댑터는 MCP의 줄 단위 JSON-RPC를 같은 정책으로 검사합니다.
- 주민등록번호는 날짜와 검증 숫자로, 카드번호는 IIN과 Luhn 검사로, 계좌번호는 앞뒤의 은행 문맥으로 확인합니다.

- 유니코드 NFKC 정규화, 숫자 사이 공백과 보이지 않는 문자 제거, 크기를 제한한 Base64 디코딩으로 우회 표현을 풀어 검사합니다.
- 정책 스키마 검증과 시크릿 분리, 만료 시간이 있는 허용 예외, Prometheus 지표, Dockerfile과 Helm 차트를 만들었습니다.

기능

HTTP 리버스 프록시

요청과 응답 본문, URL, SSE 이벤트까지 검사합니다

stdio MCP 어댑터

MCP 서버를 자식 프로세스로 띄워 양방향으로 검사하고, 막으면 JSON-RPC 오류를 돌려줍니다

한국 개인정보 탐지

주민등록번호 검증 숫자, 카드 Luhn, 계좌 은행 문맥으로 확인합니다

보호값 레지스트리

조직이 정한 기밀값을 해시로만 저장해 대조합니다

감사 로그와 지표

원래 값 없이 JSON Lines로 남기고 Prometheus로 보냅니다

시연

terminal

```
$ python -m pytest -q
.......................................................................  [ 21%]
..................s.....................................................  [ 42%]
.......................................................................  [ 64%]
.......................................................................  [ 85%]
.........................................                                [100%]
334 passed, 1 skipped in 3.26s
```

python -m pytest -q 실행 결과

기술

Python, FastAPI, uvicorn, httpx	프록시 서버와 상위 서비스 전달
regex	시간 제한이 있는 정규식으로 사용자 규칙 처리
Docker, Helm, k3d	컨테이너 이미지와 로컬 Kubernetes 배포

GitHub Actions	린트, 테스트, 정책 평가, 이미지 빌드

함께 쓰고 고친 과정

우회 사례와 테스트 초안은 Claude Code와 Codex의 제안을 참고했습니다. 탐지와 정책 기준, 정규화 방식, 구현 검토와 최종 반영은 직접 판단했습니다.

검증 방법

우회 기법을 넣은 합성 데이터로 정규화 전후의 탐지 수와 오탐을 비교했습니다. 합성 데이터는 민감 사례와 정상 사례로 나눴습니다. 프록시를 거칠 때 늘어나는 지연은 로컬에서 같은 요청을 짝지어 비교했습니다.

CASE-05 • 도구 사례 • 2026.09–2026.10

EvidScope, AI 에이전트의 행동을 감사 증거로 남기는 가시성 파일럿

AI 에이전트가 했다고 보고한 일과 도구 기록이 실제로 남긴 결과, 그때의 권한과 정책, 참고 문서 버전을 대조해, 감사 담당자가 제품 밖에서도 검증할 수 있는 증거로 남기는 로컬 파일럿입니다.

역할

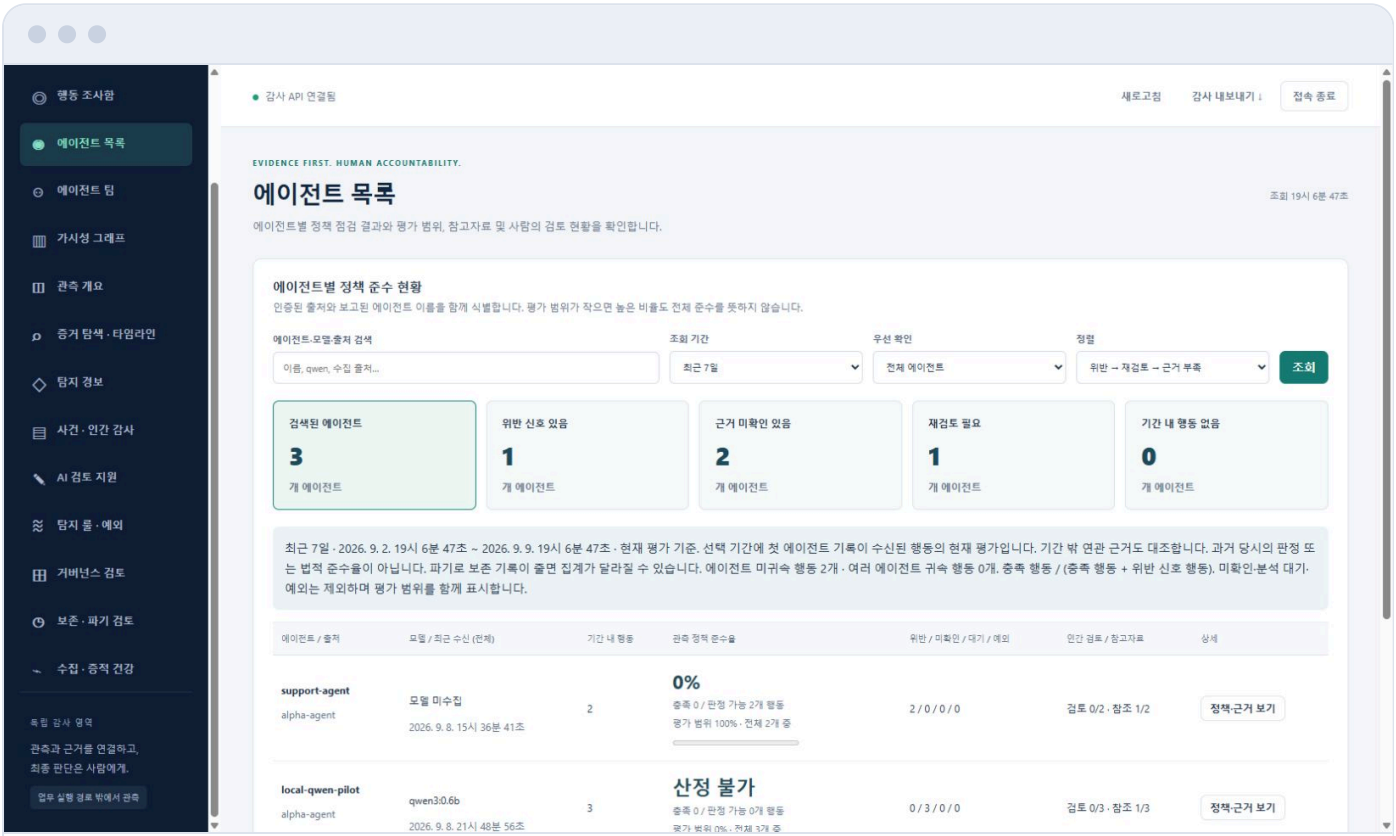
문제 정의, 설계 원칙과 조건 결정, 역할을 나눈 AI 에이전트들의 결과 검토와 공개 판단

직무

보안, AI 보안, DevOps

저장소

github.com/ki0915/EvidScope



에이전트 목록 에이전트마다 정책 평가 비율 옆에 평가 범위를 함께 보여 주고, 평가할 근거가 없으면 산정 불가로 표시합니다.

처리 흐름

01 **수집**
AI 에이전트의 행동을 OpenTelemetry로 받고 출처를 HMAC으로 확인합니다

02 **접수**
이벤트를 암호화해 원장과 서명 체크포인트에 함께 저장합니다

03 **투명성 로그**
Merkle 트리 체크포인트를 외부 위트니스가 함께 서명합니다

04 **분석**
규칙 기반 판정을 저장 프로세스가 다시 계산해 같을 때만 서명합니다

05 **사람 검토**
감사 화면에서 근거를 대조하고 판단과 보완 과제를 남깁니다

해결한 문제

문제	쓴 것	효과
요청마다 원장 전체를 다시 검증해 접수가 밀림	이미 검증한 지점 이후 변경분만 검증	**지속 구간 접수가 모두 처리됨**
감사자가 거버넌스를 바꿀 수 있던 권한 결함	공통 권한 경계에서 reviewer나 admin 역할 요구	**권한 없는 변경 요청이 모두 거부됨**
키 없이 기록을 지우거나 되돌려도 알 수 없음	Merkle 체크포인트와 외부 위트니스 공동서명	**되돌림을 찾으면 시작을 거부**
익명 로그인 요청이 몰리면 로그인이 잠김	로그인 흐름 상태를 암호화한 쿠키로 옮김	**잠금이 생기지 않음**
로컬 모델 초안이 실제 기록을 인용했지만 주장을 뒷받침하지 않음	성공으로 세지 않고 반려 기록과 원본 초안을 함께 보존	**근거가 약한 초안이 채택되지 않음**

적용 전과 후

지속 구간 접수	− ~~600건 중 84건~~ + **600건 모두**
지속 구간 응답 p95	− ~~약 6초~~ + **206.81ms**
감사자의 거버넌스 변경 요청	− ~~받아들여짐~~ + **30건 모두 거부, 서명 상태 그대로**

배경

풀려던 문제

AI 에이전트가 업무 도구를 직접 호출하면 운영자에게 남는 것은 AI의 결과 보고와 여기저기 흩어진 도구 로그입니다. 에이전트가 했다고 말한 행동이 도구의 실제 기록과 같은지, 그때 어떤 권한과 승인, 정책, 참고 문서 버전으로 행동했는지 확인하기 어렵습니다. 기록이 없을 때 그 사실이 드러나지 않으면 통과한 것처럼 보입니다.

AI 거버넌스를 하려면 먼저 AI의 행동이 보여야 한다고 보고, 행동 가시성과 감사 증거를 출발점으로 잡았습니다. 대상은 AI 서비스 운영자, 보안 담당자, 감사와 거버넌스 담당자입니다.

문제를 확인한 방법

합성 부하 시험을 돌리자 요청마다 서명 원장 전체를 다시 검증하느라 접수가 밀렸습니다.

실제 HTTP 요청으로 권한을 검토하다가, 읽기 권한만 있어야 할 감사자가 거버넌스 시스템과 평가, 과제를 바꿀 수 있는 결함을 찾았습니다.

고른 설계와 이유

AI 실행을 승인하거나 막는 제품이 아니라, 기존 SIEM과 추적 도구, 정책 엔진이 만든 기록을 감사 근거로 묶는 증거 계층으로 범위를 정했습니다.

감사 대상이 감사 결과를 바꾸지 못하게 했습니다. 업무 AI는 자기 범위의 기록을 제출만 할 수 있고, 증거 조회와 삭제, 규칙과 서명 키 변경 권한은 받지 못합니다.

AI의 자기보고, 인증된 서비스의 기록, 독립 도구의 결과를 서로 다른 상태로 저장하고, 기록이 없으면 통과가 아니라 미확인으로 남겼습니다. 근거가 없는 항목은 준수로 세지 않습니다.

수집과 집계, 정책 대조, 서명에는 LLM을 쓰지 않고, 로컬 모델은 사람이 고른 사건의 검토 초안만 쓰게 했습니다.

직접 설계하고 만든 것

- 문제와 우선순위를 정하고, 로컬 모델 실증, Kubernetes 분산, 역할별 AI 에이전트, 추가 API 비용 0원이라는 조건을 결정했습니다.
- 설계와 독립 감사, 핵심 구현, 화면을 서로 다른 AI 에이전트에 나눠 맡기고, 결과를 검토해 공개 범위를 판단했습니다.
- 시스템을 수집, 저장과 서명, 규칙 분석, 감사 화면의 네 프로세스로 나누고, 최종 인증과 서명 원장은 저장 프로세스만 갖게 했습니다.
- 원장 재검증 병목은 이미 검증한 지점 이후의 변경분만 검증하도록 바꾸고, 변조를 찾으면 멈추는 원칙은 유지했습니다.
- 감사자 권한 결함은 공통 권한 경계에서 reviewer나 admin 역할을 요구하도록 고쳤습니다.
- 한국 인공지능 기본법의 의무를 요구사항으로 옮기고, 법 적용과 충분성 판단은 사람이 내리게 했습니다.

기능

행동 대조

AI의 자기보고, 도구 결과, 당시 권한과 승인, 참고 문서 버전을 나란히 놓고 충돌과 미확인을 표시합니다

에이전트 목록

에이전트마다 정책 평가 비율 옆에 평가 범위를 함께 보여 주고, 근거가 없으면 산정 불가로 표시합니다

가시성 그래프

시간별 AI 수신 건수를 도구 성공, 도구 실패, 결과 미확인, 수집 공백으로 나눠 봅니다

사람 검토와 과제

감사 화면에서 근거를 대조해 판단과 보완 과제를 남기고, 결과를 확정하는 변경은 다른 사람이 승인합니다

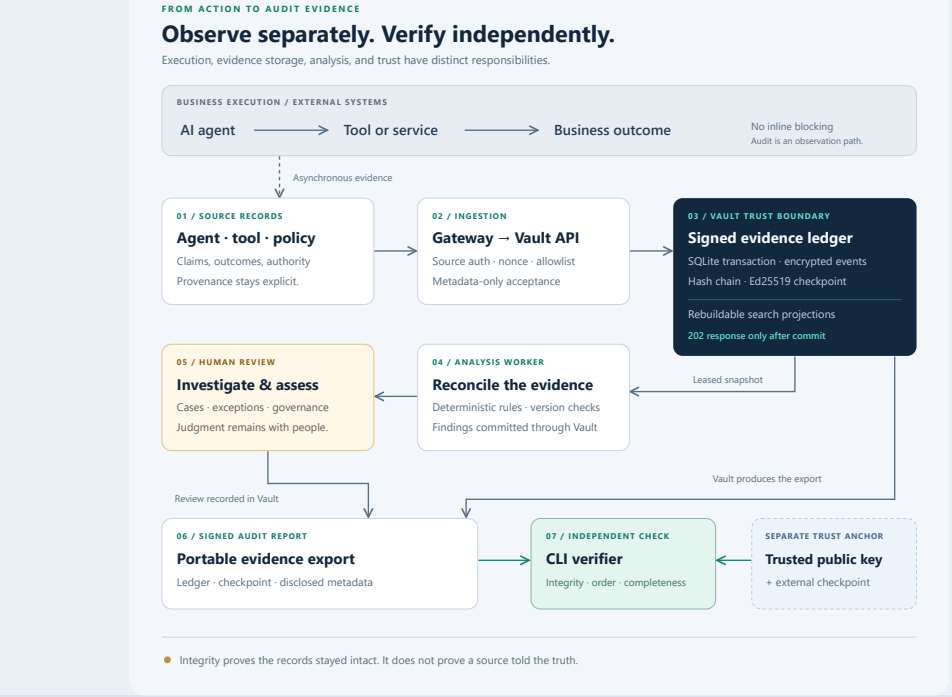
독립 검증

서명된 보고서와 원장을 내보내 별도 공개키로 제품 밖에서 검증합니다

AI 검토 초안

로컬 모델이 사람이 고른 사건의 초안을 쓰고, 사람이 채택하거나 반려합니다. 기본으로 꺼져 있습니다

시연

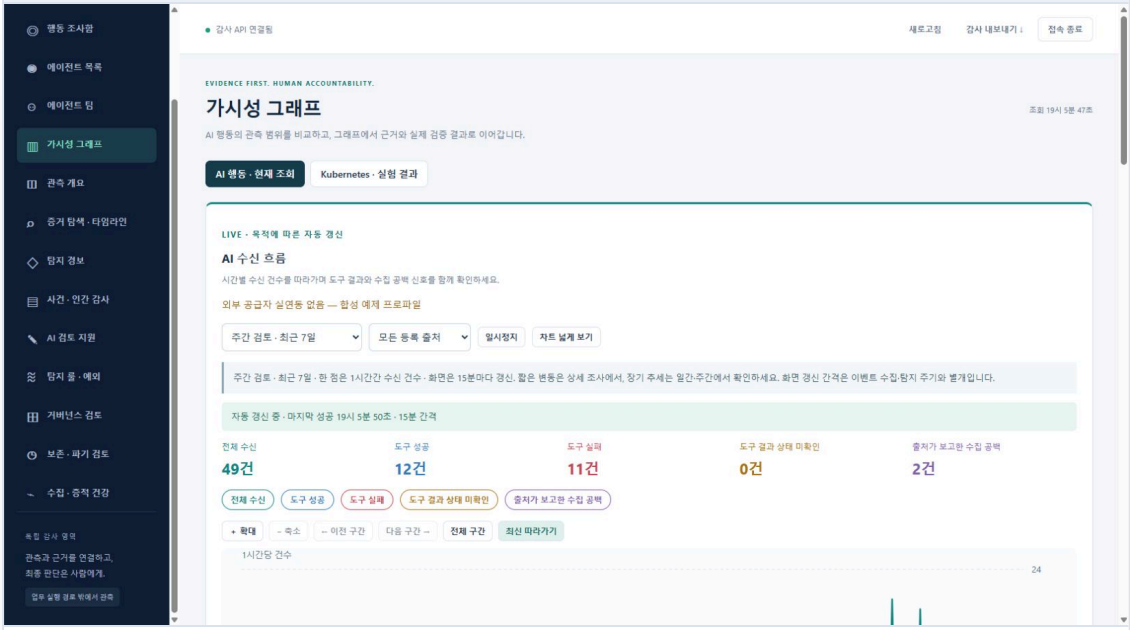


증거 흐름

terminal

```
$ npm run test:finance
✓ event metadata validates context identifiers and bounded or explicitly unknown clock error (2.2397ms)
✓ authority uses issuance time and policy known at execution, including late-arriving evidence (0.4219ms)
✓ human approval issued after execution or under another policy never satisfies prior review (0.2679ms)
✓ human oversight review requires authority provenance and system, model and policy binding (0.4688ms)
✓ self-reported delegation cannot substitute for independent parent-scoped authority (0.8162ms)
✓ stop confirmation matches the requested target and follows the request (0.8194ms)
ℹ tests 61
ℹ suites 0
ℹ pass 61
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 4526.8516
```

npm run test:finance 실행 결과



가시성 그래프 시간별 AI 수신 건수를 도구 성공, 도구 실패, 결과 미확인, 출처가 보고한 수집 공백으로 나눠 봅니다(합성 예제 데이터).



AI 검토 초안 제출 시점의 근거, 로컬 모델이 쓴 초안, 사람의 검토를 나란히 두고 근거가 주장을 뒷받침하지 않는 초안은 반려합니다.

기술

Node.js, SQLite	외부 의존성을 줄인 파일럿 서버와 원장
Ed25519, AES-256-GCM, HMAC	서명, 암호화, 출처 확인
OpenTelemetry, OCSF	수집 표준과 SIEM 연동 형식
C2SP 투명성 로그	외부 위트니스 공동서명

검증 방법

같은 합성 시나리오로 수정 전후의 부하 시험 결과를 비교했습니다. 권한 결함은 권한 없는 변경 요청 30건이 모두 거부되고 서명 상태가 바뀌지 않는 것을 회귀 시험으로 고정했습니다.

CASE-06 • 보안 사례 • 2023.04–2025.03

서버 7대 통합과 여러 겹의 백업 설계

노후 서버 7대를 Hyper-V로 합치며 외부에서 바로 들어오던 원격 접속을 없애고, 스냅샷 하나에 기대던 복구를 여러 겹의 백업으로 바꿨습니다.

역할

서버 점검, 가상화 재설계와 이관, 백업과 이중화 설계, 절차 문서화

직무

DevOps, 인프라, 보안

대응 흐름

01 **점검**

서버마다 역할과 외부 노출 상태를 확인했습니다

02 **통합**

Windows Server 2022와 Hyper-V로 합쳐 옮겼습니다

03 **접근 정리**

인바운드는 기본 차단하고 원격 접속은 내부망 경로로만 열었습니다

04 **복구 수단 나누기**

변경 제한 백업, AWS 서버 이미지, MSSQL 복제, 이중화 서버

05 **문서화**

담당자가 없어도 따라 할 수 있게 복구 절차를 남겼습니다

전과 후

복구 수단	−	~~Hyper-V 스냅샷 하나~~
	+	**변경 제한 백업, AWS 서버 이미지, MSSQL 복제**
외부 원격 접속	−	~~웹 서버와 DB에 바로 접속~~
	+	**내부망 경로로만 접근**
API 장애	−	~~이중화 없음~~
	+	**실패하면 이중화 서버로 전환**

CASE-07 • 운영 사례 • 2023.04–2025.03

5성급 호텔 서버의 디스크 고갈 장애와 재발 방지

배포 때마다 쌓인 Docker 이미지와 중지된 컨테이너가 디스크를 채워 서비스가 멈춘 장애를 복구하고, 주기 정리를 자동화해 같은 원인의 장애를 막았습니다.

역할

장애 원인 확인, 정리와 자동화, 운영 문서 작성

직무

DevOps, 인프라

대응 흐름

01 **장애 확인**

서비스가 멈춰 디스크가 가득 찬 것을 확인했습니다

02 **원인 찾기**

배포마다 남은 이미지와 중지된 컨테이너를 찾았습니다

03 **복구**

쓰지 않는 이미지와 컨테이너를 지우고 서비스를 다시 올렸습니다

04 **자동화**

같은 정리를 cron으로 주기 실행했습니다

05 **재현과 가드**

퇴사 뒤 개인 랩에서 장애를 재현하고 단계적 정리 가드를 만들었습니다

전과 후

디스크 정리	+ cron으로 주기 실행
재현 실험의 배포	− ~~14회 중 7회 실패~~
	+ 14회 모두 성공

기술 문서 • 2026년 개인 저술

Docker 컨테이너 보안 가이드 저술

위험도와 구현 비용을 기준으로 Docker 컨테이너 보안 대응을 다시 정리한 2026년 개인 기술 문서입니다.

역할	저장소
작성자	github.com/ki0915/Docker-Security-Guide

개요

OWASP Docker Top 10, CIS Docker Benchmark, NIST SP 800-190을 참고해 D01부터 D10까지의 대응 방법을 위험도와 구현 비용에 따라 재구성했습니다. 항목마다 설정 코드 예와 설명을 붙였습니다.

프로젝트

TryHackMe 모의 침투 실습과 보고서

허가된 워게임 환경에서 모의 침투를 연습하고, 공격 경로와 개선 방안을 보고서로 정리한 학습 기록입니다.

개요

TryHackMe의 Jr Penetration Tester 경로와 웹 해킹 모듈을 실습했습니다. 정찰부터 침투, 권한 상승, 개선 권고까지를 OSCP 보고서 양식으로 작성하며 보고서 쓰는 연습을 했습니다.

개인 학습 • 2023년 개인 프로젝트

WIKI

게시판과 권한별 접근 제한, 실시간 채팅을 갖춘 위키 서비스를 프론트엔드부터 백엔드까지 직접 만든 2023년 개인 프로젝트입니다.

역할	저장소
프론트엔드·백엔드 개인 개발	github.com/ki0915/wiki-backend

개요

TypeScript와 Express로 API 서버를, Angular로 클라이언트를 나눠 만들었습니다. 게시글 CRUD와 파일 업로드, JWT 인증과 bcrypt 비밀번호 해시, 역할별 권한을 구현했고, Socket.IO 채팅을 직접 붙였습니다.

개인 학습 프로젝트 • 2022년

Carrot 클론

상품을 올리고 둘러보는 거래 서비스를 본떠 화면과 서버를 연결해 본 2022년 학습 프로젝트입니다.

역할

React 프론트엔드·Node.js 백엔드 개발

개요

화면은 React와 Axios, Material UI로, 서버는 Express와 MySQL, Sequelize 기반 MVC 구조로 만들었습니다. 두 부분을 연결해 상품 조회, 상세 보기, 등록 기능을 구현했습니다. 채팅 화면은 더미 데이터로 화면 구성만 만들었습니다.

학습용 개인 프로젝트 • 2021.09–2022.01

Kiosk

2021년 9월부터 2022년 1월까지 주문 키오스크의 상품 선택부터 주문 전송까지 만들어 본 학습용 개인 프로젝트입니다.

역할

개인 개발

개요

C# WPF로 주문 화면을, Node.js와 Express, MSSQL로 서버를 만들었습니다. 상품 목록과 장바구니, 관리자용 상품 관리를 구현하고, 주문 금액은 서버로 보내 검증하게 했습니다. 회사에서 개발한 병원 키오스크 CMS와는 별개의 프로젝트입니다.